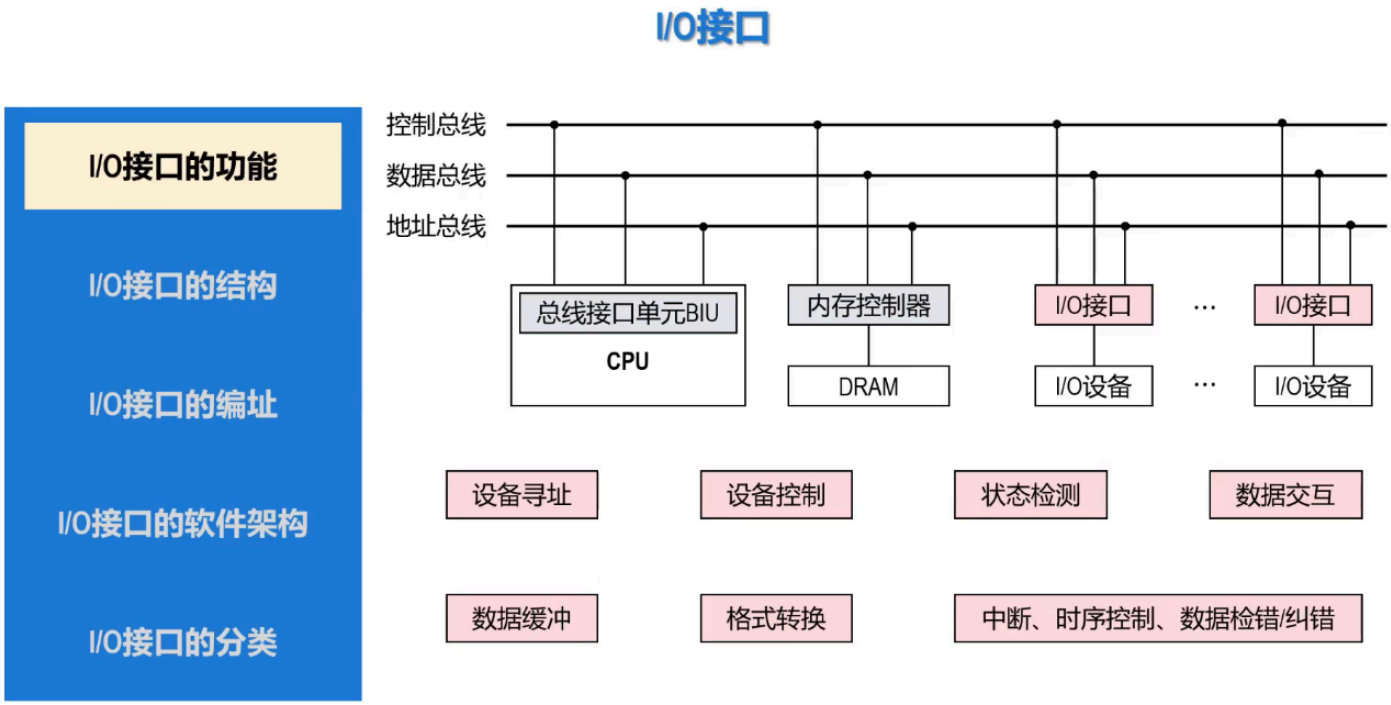
8.3 I/O 接口

考纲定位：计算机组成原理 · 第 8 章 I/O 系统（8.3）｜重要度 ★★★★｜常见考法：选择题（接口 vs 端口、DBR/DCR/DSR、统一/独立编址、指令与控制信号差异），偶与 OS 的态切换交叉考

一句话核心：I/O 接口是设备与总线之间的适配层，向上提供可被 CPU 寻址的端口，向下把端口内容翻译成设备需要的电气信号

一、I/O 接口的功能



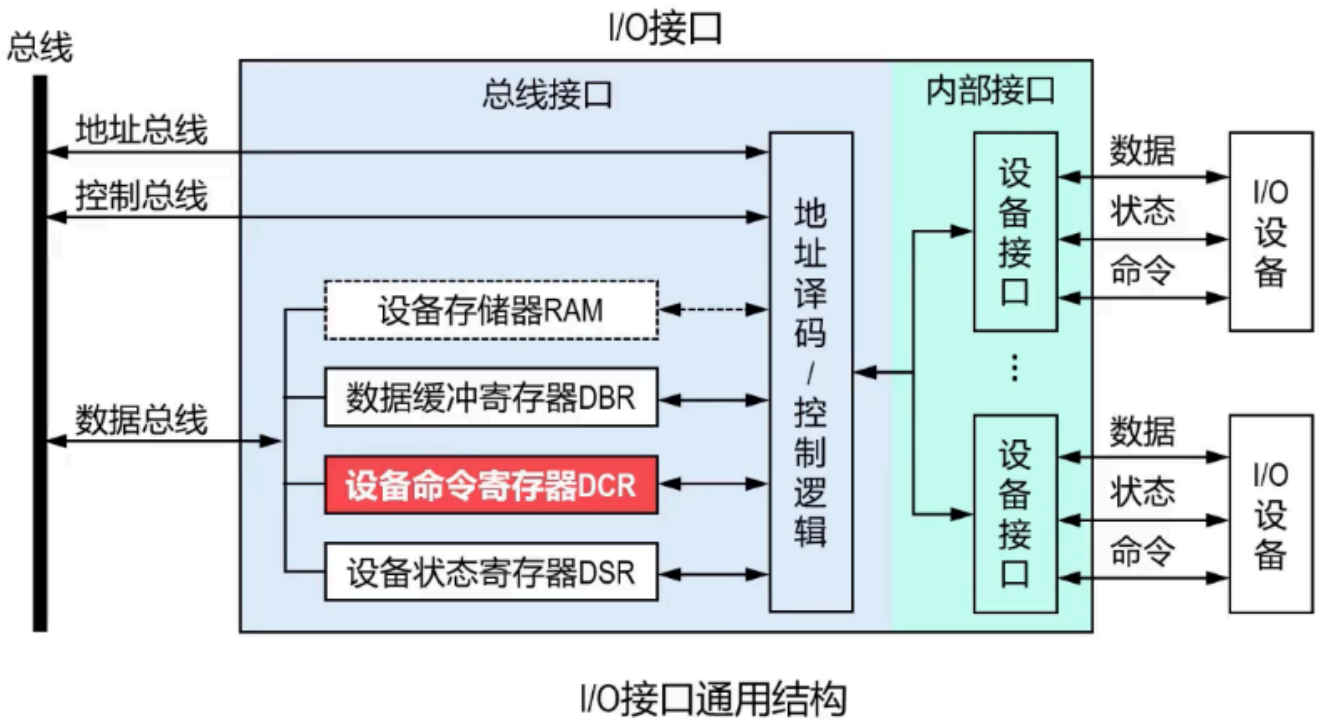
I/O 设备需要 I/O 接口才能挂载到总线上

功能	具体做什么
设备寻址	接收 CPU 送来的地址，选中本接口及接口内的某个端口
设备控制	接收 CPU 下发的命令，控制设备启动、停止、寻道、选通等动作
状态检测	把设备的忙、就绪、错误等状态提供给 CPU 读取
数据交互	在 CPU/主存与 I/O 设备之间传送数据
数据缓冲	用 DBR 等缓冲寄存器解决 CPU 与设备的速度匹配
格式转换	完成串/并转换、电平转换、数据格式转换等
中断与时序控制	产生中断请求，控制接口与设备的工作时序
数据检错/纠错	对传送过程中的数据进行检错，必要时纠错

- 不是每个接口都实现全部功能；具体实现取决于接口类型与设备需求

- 408 常考“接口的作用是什么”，答题时优先抓：寻址、控制、状态、数据缓冲、格式转换

二、I/O 接口的结构



I/O接口通用结构

图以中间竖着的“地址译码/控制逻辑”为分界线，分成两个世界：

分区	面向谁	特点
总线接口（蓝色区，左侧）	系统总线/CPU	CPU 可见，里面的寄存器就是“I/O 端口”，可被 IN/OUT 指令寻址访问
内部接口（绿色区，右侧）	I/O 设备	CPU 不可见，是设备侧的驱动逻辑，负责把寄存器内容转成设备真正需要的电气信号

核心概念辨析：接口 ≠ 端口。接口是整块适配电路；端口是接口中可被 CPU 寻址访问的寄存器（DBR、DCR、DSR 等）。一个接口包含多个端口，每个端口有自己的地址（端口地址）

2.1 四个核心寄存器

部件	全称	数据流向	作用	类比
DBR	数据缓冲寄存器	双向	CPU 与设备之间数据的中转站，解决速度匹配问题	快递柜
DCR	设备命令寄存器	CPU→写	存放 CPU 下发的操作命令（启动、停止、寻道、选通等）	遥控器按键面板
DSR	设备状态寄存器	CPU→读	反映设备当前状态：忙位、就绪位、错误位等	会议室门口的“占用中”牌

部件	全称	数据流向	作用	类比
设备存储器 RAM	—	双向	虚线框 = 可选部件，仅部分设备有（如显卡显存、磁盘盘上缓冲）	附加仓库

- 读写方向不同：DCR 只写、DSR 只读、DBR 双向，这是判断“CPU 对某端口是读是写”类选择题的依据
- 虚线框的含义：设备 RAM 是可选项，出题人常拿“图中虚线框表示什么”做文章
- DCR 在图中标红，是因为它是程序查询方式与中断方式的指挥入口，CPU 对设备的所有控制意图都从这里下发

2.2 地址译码/控制逻辑

它是整个接口的“调度中心”，只与地址总线、控制总线相连，接线方式本身就是考点：

- 地址总线送来地址 → 译码器判断：① 本接口是否被选中（片选）；② 选中的是接口内哪个端口（DBR、DCR 还是 DSR）
- 控制总线送来读/写命令 → 决定数据流向（CPU 读 DSR？CPU 写 DCR？）
- 数据总线不接译码器，直接接数据类寄存器（DBR 等）——地址线管“找谁”，数据线管“传什么”，控制线管“干什么”，三条总线各司其职

2.3 设备接口与三类信息线

每个“设备接口”通过三种线与 I/O 设备通信，方向各不相同：

信息线	方向	内容
数据	双向	真正传输的有效数据
状态	设备 → 接口	设备向接口汇报（忙/就绪/错误），最终落入 DSR 供 CPU 读
命令	接口 → 设备	接口把 DCR 中的命令翻译成设备动作

图中的“⋮”表示一个 I/O 接口可带多个设备接口，即一个接口可以管理多台同类设备

2.4 把流程串起来：以“查询方式打印一个字符”为例

1. 查状态：CPU 经地址总线选中 DSR，读出忙位，判断打印机是否空闲
2. 送数据：CPU 把字符写入 DBR，DBR 内部再传给设备接口
3. 下命令：CPU 写 DCR，发出“打印”命令
4. 执行与反馈：设备接口把命令 + 数据送到打印机；打印机一边打印，一边把新状态经状态线写回 DSR；CPU 回到第 1 步循环查询，直到完成

可见整个查询、中断、DMA 三种 I/O 方式，本质都是围绕 DBR、DCR、DSR 这三个端口做文章：查询方式是反复读 DSR，中断方式是设备就绪后主动通过接口发中断请求，DMA 方式是 DBR 与内存直接通路传输

三、I/O 接口的编址

统一编址和独立编址的本质区别只有一个：I/O 端口和主存是否共用同一个地址空间。统一编址下端口“就是”一个内存地址；独立编址下端口是独立空间里的“另一个世界”，由此派生出指令、译码、控制信号的整套差异

3.1 两种编址方式对比

对比维度	统一编址（存储器映射 I/O，MMIO）	独立编址（I/O 映射，PMIO）
地址空间	I/O 端口与主存共用一个地址空间，从主存空间划出一段给端口	I/O 端口与主存分别独立编址，两个地址空间互不相干，地址可以重叠
地址含义	同一地址要么对应主存单元、要么对应端口，二者不能同时存在	同一地址值在两个空间各有一个，靠访问方式区分指向谁
访问指令	直接使用通用访存指令（LOAD/STORE、MOV 等），无需专用 I/O 指令	必须使用专用 I/O 指令（如 x86 的 IN/OUT），地址码字段给出端口号
地址线	与主存一致，需全宽地址线	端口数少，只需少量低位地址线
译码速度	地址位数多，译码电路复杂，速度较慢	地址位数少，译码简单，速度更快
CPU 控制信号	只需一组存储器读/写信号（如 MEMR/MEMW），无需额外区分	需提供两组控制信号：存储器读/写 + I/O 读/写（如 IOR/IOW），以区分当前访问哪个空间
对端口的运算	端口可像内存一样直接参与算术逻辑运算（如 ADD [端口], 1）	端口数据必须先 IN 到寄存器，运算后再 OUT 回去，不能直接运算
主存容量影响	端口占用主存地址空间，使可用主存容量减少	不占主存空间
程序可读性	程序中 I/O 操作与访存操作混在一起，不清晰	I/O 指令显眼，程序清晰易查
典型架构	ARM、MIPS、PowerPC 等 RISC 处理器（只有这一种方式）	x86 传统上采用（同时兼容 MMIO）

3.2 关键机制：统一编址下地址冲突怎么办

设某机主存地址空间为 `0x0000 ~ 0x7FFF`，划出 `0x8000 ~ 0x80FF` 给 I/O 端口：

- 地址 `0x1234` → 落在主存区 → CPU 访问的是内存
- 地址 `0x8050` → 落在 I/O 区 → CPU 访问的是端口（比如某设备的 DBR）
- CPU 对两者发出的指令和信号完全一样，是硬件译码电路根据地址范围判断该把这次访问路由给主存还是路由给 I/O 接口
- 程序员的责任是确保不把端口地址当主存用（否则写入 `0x8050` 不是存数据，而是改了设备寄存器）

独立编址则没有这种冲突问题——地址 `0x8050` 在两个空间里各有一个，靠指令类型区分：MOV 访问的是主存，IN/OUT 访问的是端口

3.3 记忆锚点：从指令反推编址方式

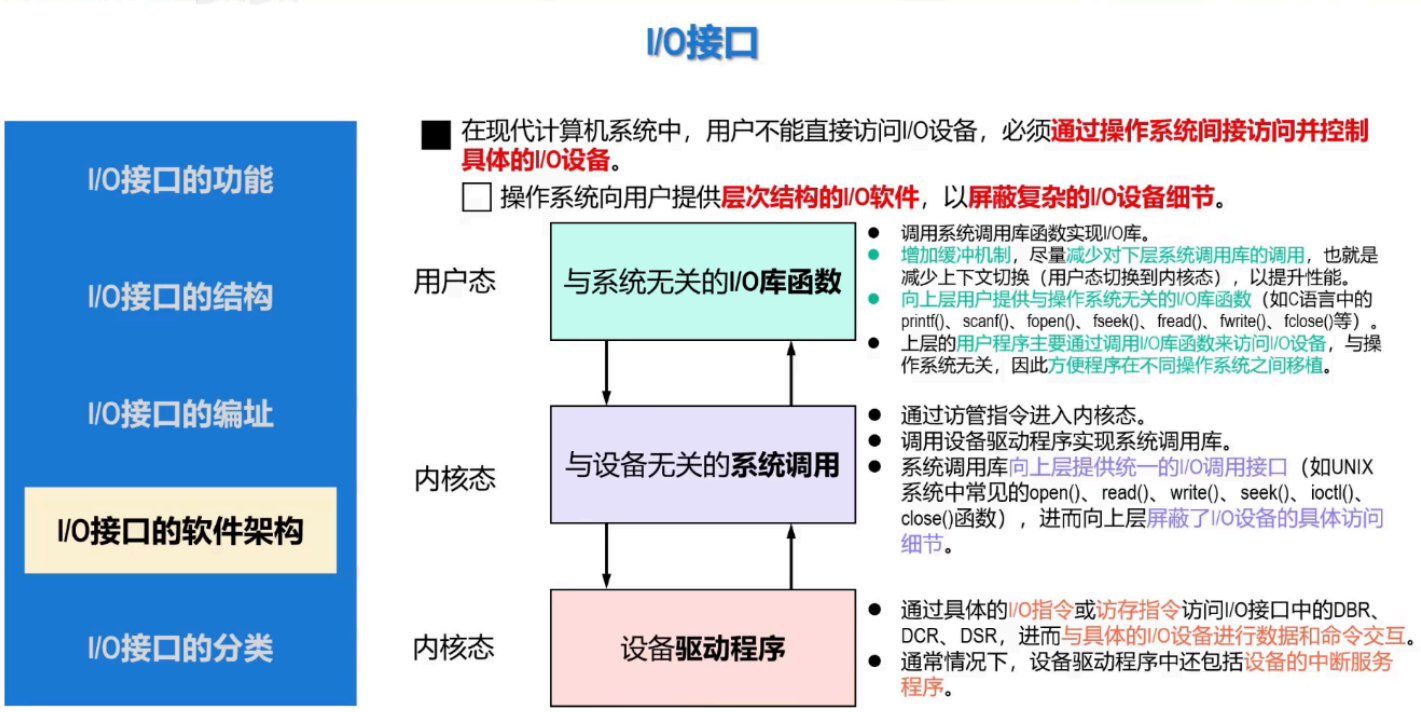
答题时一个稳准的判断口诀——看指令定编址：

- 题目说“用访存指令（MOV/LDR/STR）访问外设” → 必是统一编址
- 题目说“使用 IN/OUT 指令访问端口” → 必是独立编址
- 题目说“CPU 提供存储器读写与 I/O 读写两组控制信号” → 必是独立编址
- 题目说“CPU 无需专门区分访存与访 I/O 的引脚” → 必是统一编址

真实世界对照：x86 两者都支持，传统设备（键盘端口 0x60、RTC 端口 0x70 等）走 IN/OUT 独立编址，现代 PCIe 设备寄存器多映射到内存空间走 MMIO；ARM、MIPS 等 RISC 架构只支持统一编址，根本没有 IN/OUT 这种指令

四、I/O 接口的软件架构（跨科衔接）

> 本节属于 OS 侧内容，保留为跨科衔接，不继续扩写；OS 视角延伸见 笔记06_IO 设备设备驱动与存储介质原理_笔记



核心立意：用户不能直接访问 I/O 设备，必须通过操作系统间接访问。目的有两个：保护（外设是共享资源，不隔离会被破坏或独占）与屏蔽（不同设备差异大，分层让上层“换设备不改代码”）

4.1 三层架构

层	运行态	代表	职责
与系统无关的 I/O 库函数	用户态	printf、scanf、fopen、fread、fwrite	增加缓冲，减少系统调用次数；提供与 OS 无关的统一接口
与设备无关的系统调用	内核态	open、read、write、seek、ioctl、close	通过访管/陷入指令进入内核；提供统一 I/O 语义，屏蔽设备差异

层	运行态	代表	职责
设备驱动程序	内核态	具体设备驱动	通过 I/O 指令或访存指令访问 DBR、DCR、DSR，通常还包含中断服务程序

4.2 状态切换三条铁律

1. 访管指令（trap）本身是非特权指令，在用户态执行；系统调用的处理过程在内核态执行——“调用在用户态，执行在内核态”
2. 用户态 → 内核态只能由中断/异常（包括 trap）触发，硬件自动完成
3. 内核态 → 用户态由操作系统执行修改 PSW 的特权指令完成

4.3 库函数 vs 系统调用 vs 普通函数

对比	库函数	系统调用	普通函数调用
运行态	用户态（内部可能封装系统调用）	内核态	用户态
是否切换状态	不一定	是	否
典型例子	printf、fread	read、write	自定义函数
开销	缓冲命中时很小	陷入内核、切换堆栈，开销大	栈上跳转，开销小

- printf/fread 是库函数，read/write 是系统调用
- 缓冲区够用时，fwrite 不一定触发 write，整个过程可能都停留在用户态
- 设备驱动程序属于操作系统内核，运行在内核态，只有驱动才能执行访问 I/O 端口的特权指令

4.4 计组 vs OS 的考点侧重

学科	侧重考点	典型问法
计算机组成原理	硬件层面：驱动如何通过 IN/OUT 或访存指令访问 DBR/DCR/DSR；三种 I/O 控制方式下驱动的行为差异	“CPU 执行什么指令访问端口”
操作系统	软件层面：三层软件结构、用户态/内核态切换、系统调用执行过程、库函数与系统调用区别、设备独立性	“下列哪个操作发生在核心态”

408 喜欢在两门课的交界处挖坑，比如“应用程序调用 read() 读磁盘的过程中，CPU 状态变换几次”，需要同时用计组的中断隐指令和 OS 的系统调用陷入两条线推理

五、I/O 接口的分类



分类角度	类 型	主要特点	应用举例
数据传送方式	并行接口	多位数据并行传送，速度快，传输距离受限	SCSI、IDE等接口
	串行接口	数据逐位传输，速度相对较慢，传输距离长	SAS、SATA、USB等接口
接口的灵活性	可编程接口	可通过编程选择多种工作方式和功能	8255A（并行接口芯片）、8250（串行接口芯片）
	不可编程接口	功能固定	地址译码器
通用性	通用接口	可供多种I/O设备使用，通用性强	USB接口（可接键盘、鼠标、磁盘等）、雷电接口
	专用接口	为某类I/O设备或功能专门设计	SATA（仅用于连接存储设备）
总线传输的通信方式	同步接口	与总线之间的信息传输由统一的时钟信号控制	PCI总线接口，SDRAM内存接口
	异步接口	与总线之间的信息传输采用应答方式控制	UART（通用异步收发器，用于串口通信）
访问I/O设备方式	多 种	直接传送、程序控制、程序中断、DMA及通道处理机	程序控制：早期简单I/O芯片； 程序中断：现代I/O控制器均支； DMA接口：DMA控制器（如8237A芯片）现代硬盘/SATA控制器； 通道：大型机中的I/O处理器

分类角度	类型	主要特点	典型例子
数据传送方式	并行接口	多位数据同时传送，速度快，传输距离短	SCSI、IDE 等
数据传送方式	串行接口	数据逐位传送，速度较慢，传输距离长	SATA、SAS、USB 等
灵活性	可编程接口	可通过编程选择多种工作方式和功能	8255A 等可编程芯片
灵活性	不可编程接口	功能固定，不能通过软件改变	简单逻辑接口
通用性	通用接口	适用于多种设备	标准总线接口
通用性	专用接口	为特定设备设计	显卡、磁盘专用接口
控制方式	程序查询/中断/DMA/通道/I/O 处理机	决定 CPU 参与数据传送的程度	见 8.4-8.12

六、易错辨析

接口与结构层

易混点	核心区分
接口 vs 端口	接口是整块适配电路；端口是接口中可被 CPU 寻址的寄存器

易混点	核心区分
DBR/DCR/DSR 读写方向	DCR 只写、DSR 只读、DBR 双向
数据总线与地址总线	地址线管“找谁”，数据线管“传什么”，控制线管“干什么”；数据总线不接译码器
设备 RAM 虚线框	可选项，不是所有接口都有；常用于显卡显存、磁盘缓冲
一个接口带多个设备	图中“:”表示一个接口可管理多台同类设备，不是只能接一台

编址层

易混点	核心区分
独立编址端口地址能否与主存相同	能。独立编址的精髓就是两套空间完全独立，地址可以重叠；`IN AL, 0x60` 与 `MOV AL, [0x60]` 访问的是不同东西
统一编址能否用 IN/OUT	不能。统一编址体系下 CPU 没有 I/O 指令，一切访问都靠访存指令
统一编址一定比独立编址慢	不能绝对化。统一编址单次端口访问可能略慢，但能用访存指令对端口做复杂运算，灵活性更高
端口数少 → 地址线少 → 译码快	这是独立编址的优点，不要用反
控制引脚差异	独立编址必须额外增加控制线或控制信号编码区分两个空间；统一编址不需要
DMA 与编址	DMA 是设备与主存的数据通路问题，与端口怎么编址无关

跨科层

易混点	核心区分
库函数 = 系统调用	不是。printf/fread 是库函数；read/write 是系统调用
trap 是特权指令	反了。trap/访管指令是非特权指令，在用户态执行；系统调用处理流程在内核态
库函数一定切换内核态	不一定。缓冲区命中时可能全程用户态
系统调用 = 普通函数调用	不是。系统调用要陷入内核、切换堆栈与状态，开销大
设备驱动在用户态	错。驱动属于内核，运行在内核态

七、考法透视

常见设错点

1. 把接口当成端口，或认为一个端口就是整个接口
2. 记反 DCR/DSR 的读写方向

3. 认为独立编址下端口地址不能与主存地址重叠
4. 认为统一编址也能用 IN/OUT 指令
5. 把“端口数少 → 地址线少 → 译码快”用到统一编址上
6. 把库函数当成系统调用，或认为库函数一定陷入内核
7. 认为 trap 是特权指令

常见题型

- 给一段场景，判断是统一编址还是独立编址
- 给指令（MOV/IN/OUT），反推编址方式
- 给接口结构图，指出 DBR/DCR/DSR 或设备 RAM 的作用
- 给 read/fread 调用过程，判断 CPU 状态切换次数
- 结合 OS，问哪些操作发生在内核态

八、知识关联

- 前置：8.1 概述、8.2 IO设备及其特性
- 后续：8.4 概述（控制方式围绕接口寄存器展开）、8.5 程序直接控制方式（查询 DSR/DBR）、8.9 中断请求（中断控制器）、8.12 DMA控制方式（DMAC 与接口）
- OS 视角：笔记06_IO 设备设备驱动与存储介质原理_笔记 的三层软件架构与驱动
- 全章导航：8.0 导航与术语

九、费曼自测

1. 接口和端口的区别是什么 → 接口是整块适配电路；端口是接口中可被 CPU 寻址的寄存器，一个接口可包含多个端口
2. DBR、DCR、DSR 的读写方向分别是什么 → DBR 双向；DCR 只写；DSR 只读
3. 统一编址和独立编址的本质区别是什么 → I/O 端口与主存是否共用一个地址空间；统一编址共用，独立编址分开且地址可重叠
4. 为什么独立编址需要额外的控制信号 → 两个地址空间可能地址重叠，必须靠 IOR/IOW 等信号区分当前访问的是主存还是端口
5. 库函数、系统调用、普通函数调用有什么区别 → 库函数在用户态但可能封装系统调用；系统调用要陷入内核；普通函数调用只在用户态栈上跳转
6. 访管指令是特权指令吗 → 不是。trap/访管指令在用户态执行，它触发陷入后，系统调用处理流程才在内核态执行